

Room Discovery App — Complete Project Summary

A classifieds-style web/mobile app: users search rooms near them by live location, list rooms with photos, contact owners, and (later) compare listings.

1. What We're Building — and What We Decided Not To

Original ask: phone/email login, multi-photo room listing, contact owner, compare rooms, live-location nearby search.

Model decision: this is a **classifieds platform** (like OLX), not a booking platform (like Airbnb). "Contact and show interest" means revealing owner contact info / logging a lead — not in-app booking or payment. If you want booking + payment later, that's a separate, heavier project, not a bolt-on.

Role decision: single user type. Anyone can list, anyone can search. No verified-landlord tier in Phase 1 — not enough volume yet to justify building KYC.

Team-size assumption: small team (1-5 engineers) → monolith architecture, not microservices. See Section 4 for why.

2. Risks Identified That Weren't in the Original Ask

These weren't requested but are effectively mandatory once you allow open listing creation and phone-based auth:

Risk	Why it matters	Where it's handled
Fake/spam listings	Anyone can list, no verification	<code>pending_review</code> default status, moderation queue (Phase 1), reports (Phase 2)
OTP/contact abuse (bots, spam-clicking)	Costs you money (SMS) and degrades trust	Redis rate limiting (Phase 1)
Stale listings (owner forgets to mark rented)	Kills user trust in search results fast	Auto-expiry + renew (Phase 3)
No booking flow ambiguity	"Contact" could've meant many things	Explicitly scoped to classifieds model

3. System Architecture

Layers (top to bottom):

1. **Client** — Web app (React) + Mobile app (React Native) + device GPS feeding coordinates for nearby search
2. **API Gateway** — single entry point, delegates token verification to Auth
3. **Auth Service** — phone OTP (custom, third-party SMS) + email login
4. **Core services** (logical split, deployed as one monolith at this stage): User Profile, Listing, Search & Geo, Contact/Interest, Compare
5. **Data layer** — PostgreSQL + PostGIS (users, rooms, geo index), Object Storage + CDN (photos), Redis (sessions, search cache, compare list, rate limits)
6. **External services** — Maps API (distance/geocoding), SMS gateway (OTP), Push notifications

Why monolith, not microservices: a 5-service split adds deployment, monitoring, and debugging overhead with zero users to justify it. The service *boundaries* in the diagram are for code organization, not a mandate to deploy separately. Split later, when a specific service actually needs to scale independently — not before.

4. Tech Stack

Layer	Choice	Reasoning
Frontend	React (web) + React Native (mobile)	Shared logic, faster to ship both with a small team
Backend	Node.js (NestJS) or Django — pick one, monolith	Either handles REST + auth + uploads fine at MVP scale; the choice barely matters this early
Database	PostgreSQL + PostGIS	Standard, proven choice for geo-indexed "nearby" queries — don't reach for a geo-specialty DB without a specific reason
Image storage	S3 or Cloudflare R2 / Azure Blob + CDN	Never store images in the database
Auth	Custom OTP (third-party SMS) + Azure AD B2C or Firebase for	See Section 5 — Azure alone can't deliver OTP to Indian numbers

Layer	Choice	Reasoning
	session/token management	
Maps/distance	Google Maps, Mapbox, or Azure Maps — not finalized , verify current pricing before choosing	Needed for accurate distance calc beyond raw geo-query
Notifications	Firebase Cloud Messaging	Free, cross-platform
Hosting	Single VPS, or a managed PaaS (Azure App Service, Render, Railway)	Kubernetes/ECS is over-engineering pre-launch
Cache / rate limiting	Redis	Reused for sessions, search cache, compare lists, and rate limits — one piece of infra, four jobs

5. Hosting on Azure — Findings

You have an Azure subscription, so this was evaluated directly:

Free tier:

- \$200 credit, 30 days
- App Service: F1 free forever (sleeps when idle) or B1S free for 12 months
- Azure AD B2C: 50,000 MAU free forever — covers your auth session/token layer
- Blob Storage: 5GB free for 12 months

Not free — budget for these from day one:

- PostgreSQL Flexible Server has **no free tier at all** (unlike Azure SQL Database). Burstable B1ms ≈ \$16-20/month including storage.

Critical finding — this shaped the whole auth design: Azure Communication Services (Azure's own SMS product) **only supports sending SMS to US, Canada, and Puerto Rico numbers**. It cannot deliver OTP to Indian phone numbers. This is why the API design uses a **custom OTP flow with a third-party SMS provider** (Twilio, MSG91, Gupshup — not chosen yet) instead of relying on Azure's hosted auth end-to-end.

Other risk: Azure has no automatic spend cap, only alerts. Set a budget alert immediately — a retry-loop bug can run up a bill before anyone notices.

Estimated MVP monthly cost (post free-tier): roughly \$30-40/month infrastructure, plus per-SMS OTP cost from the third-party provider.

6. Database Schema (Phase 1)

File: `schema.sql` (already delivered). PostgreSQL 15+ with PostGIS.

Tables:

- `users` — linked to external auth provider ID, phone/email, at least one contact method required
- `rooms` — listing data, `location geography(Point, 4326)` for geo queries, `status` defaults to `pending_review` (moderation gate)
- `room_images` — URLs only (never binary), one enforced primary image per room
- `interests` — contact/lead log, unique constraint per (room, seeker) pair to block duplicate-click spam

Geo indexing: GIST index on `rooms.location`. This is what makes "nearby" search fast — without it, every radius query is a full table scan. Nearby search query uses `ST_DWithin` (radius filter) and the `<->` operator (nearest-first ordering), both using the GIST index.

Not yet built — needed for Phase 2/3: `reviews`, `reports`, `saved_searches` tables. Noted but not designed in detail yet.

7. Rate Limiting Design

Files: `rate_limiting.sql`, `rateLimitMiddleware.js` (already delivered).

Problem: a plain `COUNT(*)` check before insert has a race condition — two simultaneous requests can both pass the same count check before either commits.

Solution — two layers:

1. **Redis (primary):** atomic `INCR` + `EXPIRE`, no race condition, checked before every interest is created. Limit: 10 interests / user / 24h.
2. **Postgres trigger (backstop):** re-checks the same limit at the database level. Should never fire in normal operation — if it does, something bypassed the Redis check and needs investigating.

Open decision, not yet made: fail-open vs fail-closed if Redis is down. Currently implemented as **fail-closed** (blocks the request) — safer against abuse, but means a Redis

outage takes down the "contact owner" feature. This is a business tradeoff, not a technical one — you need to confirm this is the right call for your priorities.

Same pattern (not yet built) is needed for **OTP request/verify rate limiting** — arguably higher priority since SMS costs money per abused request.

8. Complete API Design (All Phases)

Full detail in `api_routes_all_phases.md` (already delivered). Summary:

Phase 1 — MVP

- **Auth:** custom OTP request/verify (rate limited), email login, session management
- **Users:** profile get/update
- **Rooms:** create (→ `pending_review`), get, update (resets to `pending_review`), soft delete, list/filter, **nearby search**, view counter
- **Room Images:** presigned upload flow, confirm, delete, set primary, reorder
- **Interests:** create (rate limited), sent/received lists, status update
- **Admin (minimal):** pending queue, approve/reject/flag

Phase 2 — after Phase 1 shows real usage

- **Compare:** Redis-backed (not a DB table — compare lists are disposable), cap 4 rooms
- **Fuller moderation:** user reports, admin report queue, auto-flag after N reports (N not yet chosen)

Phase 3 — after Phase 1 shows retention

- **Availability toggle:** lightweight route that flips active/rented **without** triggering full re-review (resolves the tension between "edits need review" and "owners need to update availability daily")
 - **Auto-expiry + renew:** listings expire after 30 days without renewal (background job, not a route)
 - **Reviews:** only from users with a verified `interests` record for that room — prevents fake reviews
 - **Saved searches + push notifications:** alert criteria stored, matching done via background job on new-listing approval, not synchronously on every request
-

9. Pros and Cons of Key Decisions

Decision	Pros	Cons
Classifieds model over booking	Much smaller build, faster to launch, matches what was actually requested	No revenue mechanism built in (no transaction fee possible without payment flow); would need a separate project to add booking later
Monolith over microservices	Simple to deploy, debug, and reason about with a small team	Will eventually need refactoring if/when specific services need independent scaling
PostgreSQL + PostGIS	Proven, standard geo-query performance; one database for both relational and geo data	Slightly more ops overhead than a fully managed NoSQL option; not serverless
Azure for hosting	You already have the subscription; Entra ID auth is generous and free at this scale	No free tier for Postgres; cannot send OTP SMS to India — forces a third-party dependency for a core feature
Redis for rate limiting	Atomic, fast, no race conditions, reuses infra you already need	Adds a dependency the "contact" feature relies on; fail-closed means an outage blocks a core action
<code>pending_review</code> as listing default	Blocks fake/spam listings from going live silently	Adds friction — legitimate listings are invisible until reviewed; slow moderation = bad first impression
Compare in Redis, not Postgres	No schema bloat for a disposable, session-scoped feature	Compare lists don't persist long-term unless you explicitly add TTL/backup logic later

10. Open Decisions — Not Yet Made, Need Your Input

These were flagged during the conversation and deliberately left unresolved because they're business calls, not engineering ones:

1. **Fail-open vs fail-closed** for rate limiting if Redis goes down.
2. **SMS provider choice** — Twilio, MSG91, Gupshup, or other, for OTP delivery to Indian numbers.
3. **Maps API provider** — Google Maps, Mapbox, or Azure Maps. Pricing wasn't confidently confirmed during research; verify current rates directly before

committing.

4. **Report threshold (N)** before a listing auto-flags in Phase 2 — needs real traffic data to set sensibly.
 5. **Listing expiry window** — 30 days was used as a placeholder in Phase 3 design, not a confirmed decision.
 6. **Minimum photo requirement** — should a listing be blockable from admin approval if it has zero photos? Leaning yes, not finalized.
-

11. Files Delivered So Far

- `schema.sql` — Phase 1 database schema with PostGIS geo indexing
 - `rate_limiting.sql` — Postgres trigger backstop for interest rate limiting
 - `rateLimitMiddleware.js` — Redis-based primary rate limit check (Node/Express)
 - `api_routes_all_phases.md` — full API route design across all three phases
 - Excalidraw system architecture diagram (rendered inline in conversation, not a file)
-

12. Recommended Next Steps

In priority order:

1. Resolve the six open decisions in Section 10 — don't let engineering start on ambiguous ground.
2. Choose the SMS provider — this blocks the entire auth flow.
3. Build Phase 1 only. Resist adding Phase 2/3 features "since you're in there anyway" — that instinct is what turns a 2-week MVP into a 2-month one.
4. Add the three missing tables (`reviews`, `reports`, `saved_searches`) to `schema.sql` only when you actually start Phase 2/3 — don't pre-build them now.
5. Set an Azure budget alert before deploying anything, given the no-spend-cap risk noted in Section 5.